

Отчет по рефакторингу кода Task Manager

1. Исходное состояние кода (old_main.py)

Файл `old_main.py` представляет собой реализацию диспетчера задач с использованием `tkinter` и `psutil`. Он позволяет просматривать список процессов, сортировать их, искать по различным критериям и завершать процессы через контекстное меню. Однако код имеет ряд проблем, которые затрудняют его чтение, поддержку и тестирование:

- **Монолитная структура:** Весь код находится в одном файле, включая глобальные переменные, функции и логику интерфейса, без четкого разделения ответственности.
- **Дублирование кода:** Например, логика завершения процессов повторяется в функциях `kill_process` и `force_kill_process` в классе `Context_Menu`.
- **Низкий уровень абстракции:** Многие функции выполняют сразу несколько задач (например, обновление данных и сортировка в `update_data`), что усложняет их переиспользование и тестирование.
- **Проблемы с именованием:** Некоторые переменные и функции имеют неинформативные имена (например, `proc` вместо более конкретного `process_info`), а глобальные переменные вроде `current_menu` используются без явного контекста.
- **Сложность поддержки:** Отсутствие классов или модулей приводит к тому, что изменение одной части кода может затрагивать другие, не связанные с ней участки.

Пример проблемного участка (из `update_data`):

```
def update_data():
    global processes_sorted, filtered_processes, is_search_active
    if is_search_active:
        root.after(2000, update_data)
        return
    processes = list(psutil.process_iter(["pid", "name", "cpu_percent",
    "memory_info", "status"]))
    processes_sorted = []
    for proc in processes:
        try:
            pid = proc.info["pid"]
            name = proc.info.get("name", "N/A")
            cpu = proc.info["cpu_percent"]
            memory = (proc.info["memory_info"].rss / 1024 / 1024) if
proc.info["memory_info"] else 0
            status = proc.info.get("status", "N/A")
            processes_sorted.append((pid, name, f"{cpu:.2f}%", f"
{memory:.2f} MB", status))
        except (psutil.NoSuchProcess, psutil.AccessDenied, KeyError):
            continue
    # Далее идет сортировка и обновление интерфейса...
```

- Проблемы: функция одновременно собирает данные, форматирует их, сортирует и обновляет интерфейс. Это нарушает принцип единственной ответственности.

2. Обратное проектирование

Для анализа структуры кода была построена упрощенная диаграмма зависимостей:

- **Глобальные переменные** (`processes_sorted`, `filtered_processes`, `current_menu`, etc.) используются повсеместно.
- **Основные компоненты:**
 - Логика интерфейса (`tree`, `state_combobox`, etc.).
 - Логика работы с процессами (`psutil`-функции).
 - Контекстное меню (`Context_Menu`).
- **Зависимости:** Все функции зависят от глобальных переменных и напрямую взаимодействуют с `tkinter`-объектами.

Проблемные участки:

- Функции сортировки (`sort_by_memory`, `sort_by_name`, etc.) дублируют логику.
- Управление контекстным меню в `Context_Menu` смешивает логику интерфейса и завершения процессов.

3. Внесенные изменения (`main.py`)

После рефакторинга код был переработан в файл `main.py`. Вот основные изменения и их обоснование:

1. Введение классов и разделение ответственности:

- Создан класс `TaskManagerApp`, который инкапсулирует всю логику приложения.
- Класс `ContextMenu` выделен для управления контекстным меню, изолируя его от основной логики.
- **Обоснование:** Это соответствует принципу единственной ответственности (SRP). Теперь каждый класс отвечает за свою область: `TaskManagerApp` — за интерфейс и данные, `ContextMenu` — за контекстное меню.

2. Устранение дублирования:

- Логика завершения процессов (`terminate` и `kill`) объединена в метод `_kill_process` класса `ContextMenu` с параметром `force`.

```
def _kill_process(self, pid, force=False):
    try:
        process = psutil.Process(pid)
        if force:
            process.kill()
            messagebox.showinfo("Успех", f"Процесс {pid} был
            принудительно завершен.")
        else:
            process.terminate()
            messagebox.showinfo("Успех", f"Процесс {pid}
            завершен.")
        self.app.filtered_processes = [proc for proc in
```

```

self.app.filtered_processes if proc[0] != pid]
    self.app.update_treeview()
except psutil.NoSuchProcess:
    messagebox.showerror("Ошибка", f"Процесс {pid} не
    существует.")

```

- **Обоснование:** Уменьшает дублирование кода и упрощает поддержку.

3. Улучшение именования:

- Переименованы методы и переменные для большей ясности, например, `kill_process` → `_terminate_process`, `proc` → `process_info`.
- Префикс `_` добавлен к внутренним методам (например, `_setup_ui`), чтобы обозначить их как приватные.
- **Обоснование:** Улучшает читаемость и отражает назначение функций/переменных.

4. Введение уровней абстракции:

- Функция `update_data` разбита на сбор данных, сортировку и обновление интерфейса. Сортировка вынесена в отдельный метод `sort_processes_by_field`.

```

def sort_processes_by_field(self, field: str, reverse: bool) ->
None:
    if self.current_sort_field == field:
        self.current_sort_order = not self.current_sort_order
    else:
        self.current_sort_field = field
        self.current_sort_order = reverse
    key_func = {
        "memory": lambda proc: float(proc[3].replace(" MB", "")),
        "name": lambda proc: proc[1].lower(),
        # ...
    }.get(field)
    if key_func:
        self.processes_sorted.sort(key=key_func,
reverse=self.current_sort_order)
        self.update_treeview()

```

- **Обоснование:** Упрощает тестирование и повторное использование кода.

5. Устранение глобальных переменных:

- Переменные вроде `processes_sorted` и `filtered_processes` стали атрибутами класса `TaskManagerApp`.
- **Обоснование:** Уменьшает побочные эффекты и делает зависимости явными.

6. Мелкие улучшения:

- Добавлены аннотации типов (например, `sort_processes_by_field(self, field: str, reverse: bool) -> None`).

- Выделены вспомогательные методы настройки интерфейса (`_setup_treeview`, `_setup_search_frame`).
- **Обоснование:** Повышает читаемость и поддерживаемость.

4. Итоговое состояние кода

Файл `main.py` стал более структурированным и модульным:

- **Структура:** Два класса (`TaskManagerApp` и `ContextMenu`) с четкими обязанностями.
- **Читаемость:** Код разбит на логические блоки, имена методов/переменных отражают их назначение.
- **Поддерживаемость:** Изменения в одной части кода (например, интерфейсе) не затрагивают другие (например, логику завершения процессов).
- **Пример итогового кода (часть `TaskManagerApp`):**

```
class TaskManagerApp:
    def __init__(self):
        self.root = tk.Tk()
        self.root.title("Task Manager")
        self.root.geometry("600x600")
        self.root.configure(background="#1e2120")
        self.current_sort_field = "memory"
        self.current_sort_order = True
        self.processes_sorted = []
        self.filtered_processes = []
        self._setup_ui()
        self.context_menu = ContextMenu(self)

    def _setup_ui(self):
        self._setup_treeview()
        self._setup_search_frame()
        self.tree.bind("<Button-3>", lambda event:
self.context_menu.create_context_menu(self.tree, event))
        self.root.after(2000, self.update_data)
```

5. Проверка корректности

После рефакторинга программа была протестирована:

- Отображение списка процессов работает корректно.
- Сортировка по всем полям (PID, Name, CPU, Memory, Status) сохраняет прежнее поведение.
- Поиск по PID, имени, состоянию и портам функционирует без изменений.
- Контекстное меню завершает процессы как ожидается.
- Окно с информацией о процессе обновляется каждую секунду и возвращает к списку при закрытии.

Вывод

Рефакторинг устранил основные недостатки исходного кода, сделав его более структурированным, читаемым и легким для поддержки. Примененные техники (разделение ответственности, устранение дублирования, улучшение именования, введение абстракций) соответствуют принципам чистого кода, сохраняя при этом полную функциональность программы.